



Lecture-7

Goal Based Agent, Problem-solving Agents, Goal Formulation and Problem Formulation

Course Teacher

Prof. Dr. Md. Ashikur Rahman Khan

Goal-Based Agent

Two kinds of Goal Based Agent

1. Problem-solving Agent:
2. Planning Agent

1. Problem-solving Agent:

What is a problem?

a question that requires an answer.

PSA is a Special type of goal based agent.

Problem-solving agents use atomic representations—

that is, states of the world are considered as wholes, with no internal structure visible to the problem solving algorithms.

Finds sequence of actions that achieve goals.

2. Planning Agent: Goal-based agents that use more advanced factored or structured representations are usually called planning agents.

Problem-Solving Agents



Four general steps in problem solving:

1. **Goal formulation: objectives that need to be achieved**
(where a goal is set of acceptable states)
2. **Problem formation: actions and states to be considered** (choose the operators and state space)
3. **Search (Sequence)**
 - **Search Strategies**
 - **Constraint Satisfaction**
4. **Solution (Action) or Execute solution:** Given the solution, perform the actions.

Prof. Dr. Md. Ashikur Rahman Khan

Goal Formulation



Goal – It is the description of desirable situations. Or **Specify the objectives to be achieved.**

Goal helps organize behavior **by limiting the objectives** that the agent is trying to achieve and hence the actions it needs to consider.

Goal formulation means ignoring all other aspects of the current state and the performance measure, and choosing a goal.

Goal formulation, based on the current situation and the agent's performance measure, **is the first step in problem solving.**

Prof. Dr. Md. Ashikur Rahman Khan

1. Goal Formulation (Contd.)



Imagine an agent in the city of Arad, Romania, enjoying a touring holiday.

The agent's performance measure contains many factors:

it wants to improve its suntan,
improve its Romanian,
take in the sights,
enjoy the nightlife (such as it is),
avoid hangovers, and so on.

- **The decision problem is a complex one involving many tradeoffs and careful reading of guidebooks.**
- **Now, suppose the agent has a nonrefundable ticket to fly out of Bucharest the following day.**

Prof. Dr. Md. Ashikur Rahman Khan

Goal Formulation (Contd.)



In that case, it makes sense for the agent to adopt the goal of getting to Bucharest.

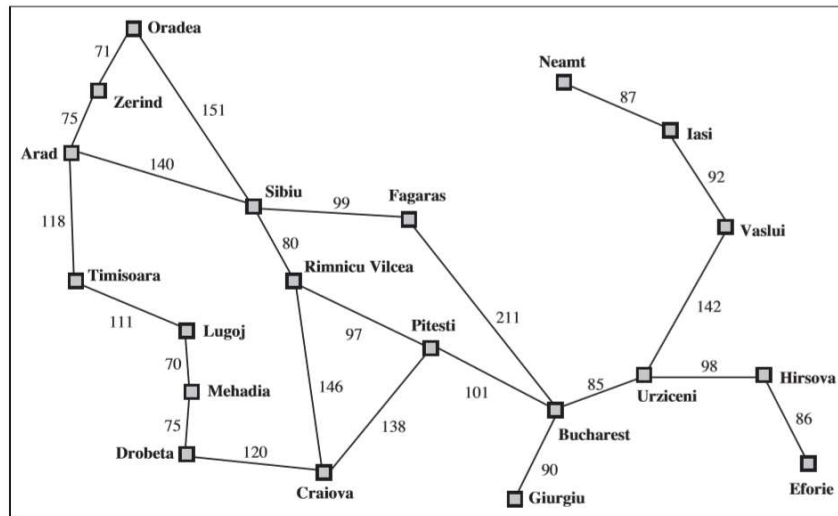
Courses of action that don't reach Bucharest on time can be rejected without further consideration and the agent's decision problem is greatly simplified.

We will consider a goal to be a set of world states—exactly those states in which the goal is satisfied. **The agent's task is to find out how to act, now and in the future, so that it reaches a goal state.**

Before it can do this, it needs to decide (or we need to decide on its behalf) what sorts of actions and states it should consider.

Prof. Dr. Md. Ashikur Rahman Khan

Goal Formulation (Contd.)

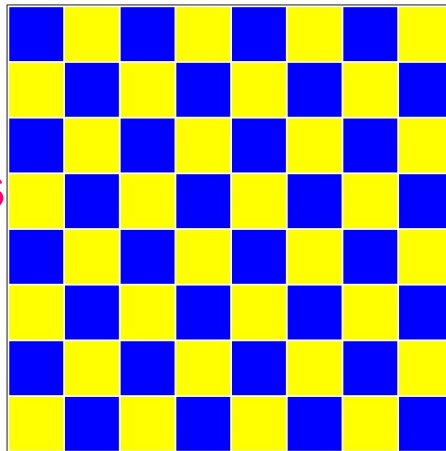


Prof. Dr. Md. Ashikur Rahman Khan

Goal Formulation (Contd.)

Example # 2

Chess



What is goal here?

In chess, the goal is to reach a state called “checkmate,” where the opponent’s king is under attack and can’t escape.

Prof. Dr. Md. Ashikur Rahman Khan

Goal Formulation (Contd.)



Example # 3

8-Puzzle

The 8-puzzle, an instance of which is shown in Figure below, consists of a 3×3 board with eight numbered tiles and a blank space. A tile adjacent to the blank space can slide into the space.

The object is to reach a specified goal state, such as the one shown on the right of the figure.

8-Puzzle

7	2	4
5		6
8	3	1

Start State

What is goal here?

Figure : A typical instance of the 8-puzzle

Problem Abstraction

8-puzzle problem

Determine goal formulation and problem formulation for 8-puzzle problem. Write a program to reach the goal of **8-puzzle problem**.

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State



What abstractions have we included here?

What abstractions have we included here?

The actions are abstracted to their beginning and final states, ignoring the intermediate locations where the block is sliding.

We have abstracted away actions such as shaking the board when pieces get stuck and ruled out extracting the pieces with a knife and putting them back again.

We are left with a description of the rules of the puzzle, avoiding all the details of physical manipulations.

Prof. Dr. Md. Ashikur Rahman Khan

Problem Formulation

2. Problem formulation

After goal formulation, the agent must do problem formulation.

Problem formulation is the process of deciding what actions and states to consider, given a goal.

Let us assume that the agent will consider actions at the level of driving from one major town to another.

Each state therefore corresponds to being in a particular town.

Problem Formulation

Thus, we have a simple “**formulate**, **search**, **execute**” **design for the agent**, as shown in Figure 3.1.

```
function SIMPLE-PROBLEM-SOLVING-AGENT(percept) returns an action
  persistent: seq, an action sequence, initially empty
              state, some description of the current world state
              goal, a goal, initially null
              problem, a problem formulation

  state ← UPDATE-STATE(state, percept)
  if seq is empty then
    goal ← FORMULATE-GOAL(state)
    problem ← FORMULATE-PROBLEM(state, goal)
    seq ← SEARCH(problem)
    if seq = failure then return a null action
  action ← FIRST(seq)
  seq ← REST(seq)
  return action
```

It first formulates a goal and a problem, searches for a sequence of actions that would solve the problem, and then executes the actions one at a time.

Figure 3.1 A simple problem-solving agent.

When this is complete, it formulates another goal and starts over.

Problem Formulation (Contd.)



A problem can be defined formally by **five components**:

- 1) **Initial state**
- 2) **Actions**
- 3) **Transition Model**
- 4) **Goal Test**
- 5) **Path Cost**

Problem Formulation (Contd.)

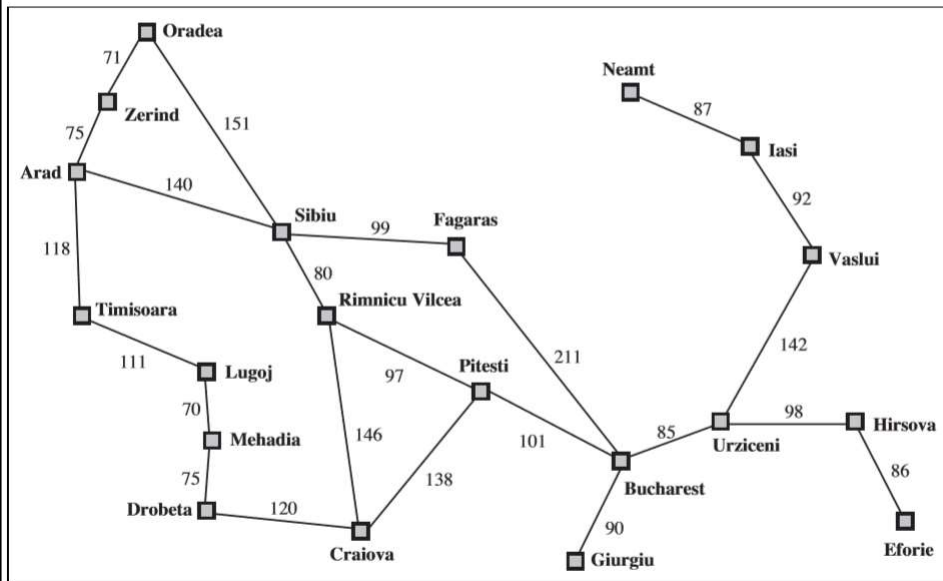


Figure 3.2: A simplified road map of part of Romania

Prof. Dr. Md. Ashikur Rahman Khan

Problem Formulation (Contd.)

i. Initial state— The initial state that the agent starts in.

For example, the initial state for our agent in Romania might be described as $In(Arad)$.

ii. Actions—A description of the possible actions available to the agent.

Given a particular state, s , $ACTIONS(s)$ returns the set of actions that can be executed in, s .

We say that each of these actions is applicable in s .

For example, from the state $In(Arad)$, the applicable actions are $\{Go(Sibiu), Go(Timisoara), Go(Zerind)\}$.

Prof. Dr. Md. Ashikur Rahman Khan

Problem Formulation (Contd.)



iii. Transition model: A description of what each action does; the formal name for this is the transition model, specified by a function **RESULT(s, a)** that returns the state that results from doing action **a** in state **s**.

We also use the term successor to refer to any state reachable from a given state by a single action. For example, we have

$$\text{RESULT}(\text{In}(\text{Arad}), \text{Go}(\text{Zerind})) = \text{In}(\text{Zerind})$$

Together, the initial state, actions, and transition model implicitly define the **state space** of the problem.

Prof. Dr. Md. Ashikur Rahman Khan

Problem Formulation (Contd.)



iv. Goal Test: The goal test, which determines whether a given state is a goal state. Sometimes there is an explicit set of possible goal states, and the test simply checks whether the given state is one of them.

The agent's goal in Romania is the singleton set $\{\text{In}(\text{Bucharest})\}$.

Sometimes the goal is specified by an abstract property rather than an explicitly enumerated set of states.

For example, in chess, the goal is to reach a state called "checkmate," where the opponent's king is under attack and can't escape.

Prof. Dr. Md. Ashikur Rahman Khan

Problem Formulation (Contd.)



v. Path Cost: A path in the state space is a sequence of states connected by a sequence of actions.

A path cost function that assigns a numeric cost to each path. The problem-solving agent chooses a cost function that reflects its own performance measure.

For the agent trying to get to Bucharest, time is of the essence, so the **cost of a path might be its length in kilometers**.

In this chapter, we assume that the cost of a path can be described as the sum of the costs of the individual actions along the path.

Prof. Dr. Md. Ashikur Rahman Khan

Types of Problem



Types of Problem

- i. Toy problems
- ii. Real-world problems

i. A **toy problem** is intended to illustrate or exercise various problem-solving methods. It can be given a concise, exact description and hence is usable by different researchers to compare the performance of algorithms.

- a. Vacuum world
- b. 8-queens
- c. 8-puzzle
- d. Missionaries and cannibals

Prof. Dr. Md. Ashikur Rahman Khan

Types of Problem



ii. A **real-world problem** is one whose solutions people **actually care about**. Such problems tend not to have a single agreed-upon description, but we can give the general flavor of their formulations.

Real World Problem

- a. Traveling Salesperson Problem (NP hard)
- b. Route finding
- c. VLSI layout
- d. Robot navigation
- e. Assembly sequencing
- f. Web searching

Prof. Dr. Md. Ashikur Rahman Khan

Toy Problem



a. Vacuum Cleaner World

Consider the **vacuum world** as first introduced as shown in Figure below.

The first example we examine is the vacuum world (first introduced in Chapter 2. See Figure 2.2.)

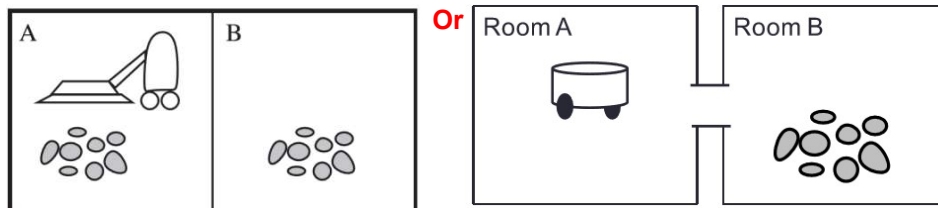


Figure: A vacuum-cleaner world with just two locations.

Prof. Dr. Md. Ashikur Rahman Khan

a. Vacuum Cleaner World (Contd.)



This can be formulated as a problem as follows:

- i. **States:** The state is determined by both the agent location and the dirt locations. The agent is in one of two locations, each of which might or might not contain dirt.

Thus, there are $2 \times 2^2 = 8$ possible world states.

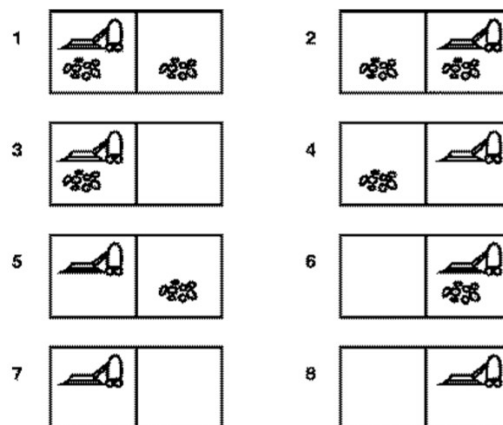
A larger environment with n locations has $n \cdot 2^n$ states.

Sl	Agent location	Dirt locations (Might or might not contain dirt)		
1	A	Clean	B	Dirt
2	A	Dirt	B	Clean
3	A	Clean	B	Clean
4	A	Dirt	B	Dirt
5	B	Clean	A	Dirt
6	B	Dirt	A	Clean
7	B	Clean	A	Clean
8	B	Dirt	A	Dirt

a. Vacuum Cleaner World (Contd.)



Consider the vacuum cleaner world.



Compared with the real world, this toy problem has discrete locations, discrete dirt, reliable cleaning, and it never gets any dirtier.

Prof. Dr. Md. Ashikur Rahman Khan

a. Vacuum Cleaner World (Contd.)



This can be formulated as a problem as follows:

- i. **Initial state:** Any state can be designated as the initial state.
- ii. **Actions:** In this simple environment, each state has just three actions: **Left**, **Right**, and **Suck**. Larger environments might also include **Up** and **Down**.
- iii. **Transition model:** The actions have their expected effects, except that moving **Left** in the leftmost square, moving **Right** in the rightmost square, and **Sucking** in a clean square have no effect. The complete state space is shown in Figure 3.3.
- iv. **Goal test:** This checks whether all the squares are clean.
- v. **Path cost:** Each step costs 1, so the path cost is the number of steps in the path.

a. Vacuum Cleaner World (Contd.)

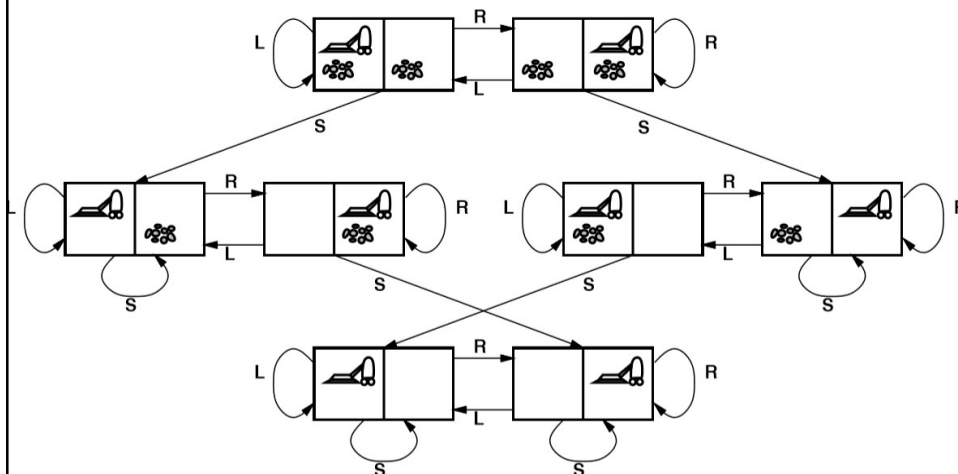


Figure 3.3: The state space for the vacuum world. Links denote actions: L = *Left*, R = *Right*, S = *Suck*.

Prof. Dr. Md. Ashikur Rahman Khan

b. 8-puzzle problem

Determine goal formulation and problem formulation for 8-puzzle problem. Write a program to reach the goal of **8-puzzle problem**.

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

b. 8-puzzle problem (Contd.)



7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

Figure 3.4: A typical instance of the 8-puzzle.

What is its formulation?

b. 8-puzzle problem (Contd.)



The standard formulation is as follows:

States: A state description specifies the location of each of the eight tiles and the blank in one of the nine squares.

i. Initial state: Any state can be designated as the initial state. Note that any given goal can be reached from exactly half of the possible initial states.

ii. Actions: The simplest formulation defines the actions as movements of the blank space **Left, Right, Up, or Down**. Different subsets of these are possible depending on where the blank is.

iii. Transition model: Given a state and action, this returns the resulting state; for example, if we apply **Left** to the start state in Figure 3.4 (above), the resulting state has the 5 and the blank switched.

b. 8-puzzle problem (Contd.)



The standard formulation (contd.)

iv. Goal test: This checks whether the state matches the goal configuration shown in Figure 3.4. (Other goal configurations are possible.)

v. Path cost: Each step costs 1, so the path cost is the number of steps in the path.

Other puzzle problem



The **8-puzzle** belongs to the family of sliding-block puzzles, **which are often used as test problems for new search algorithms in AI**. This family is known to be **NP-complete**, so one does not expect to find methods significantly better in the worst case than the search algorithms described in this chapter and the next.

The 8-puzzle has $9!/2=181,440$ reachable states and is easily solved.

The 15-puzzle (on a 4×4 board) has around 1.3 trillion states, and random instances can be solved optimally in a few milliseconds by the best search algorithms. The 24-puzzle (on a 5×5 board) has around 10^{25} states, and random instances take several hours to solve optimally.

Prof. Dr. Md. Ashikur Rahman Khan

c. Donald Knuth Problem



Problem devised by Donald Knuth

A toy problem was devised by Donald Knuth (1964) and illustrates how infinite state spaces can arise.

Knuth conjectured that, starting with the number 4, a sequence of factorial, square root, and floor operations will reach any desired positive integer.

For example, we can reach 5 from 4 as follows:

$$\left\lfloor \sqrt{\sqrt{\sqrt{\sqrt{\sqrt{(4!)!}}}}} \right\rfloor = ?$$

What is its formulation?

Prof. Dr. Md. Ashikur Rahman Khan

c. Donald Knuth Problem (Contd.)



$$\left\lfloor \sqrt{\sqrt{\sqrt{\sqrt{\sqrt{(4!)!}}}}} \right\rfloor = 5.$$

The problem definition is very simple:

States: Positive numbers.

i. Initial state: 4.

ii. Actions: Apply factorial, square root, or floor operation (factorial for integers only).

iii. Transition model: As given by the mathematical definitions of the operations.

iv. Goal test: State is the desired positive integer.

v. Path costs: 0 (we are only interested in the solution).

Prof. Dr. Md. Ashikur Rahman Khan

Other Problems



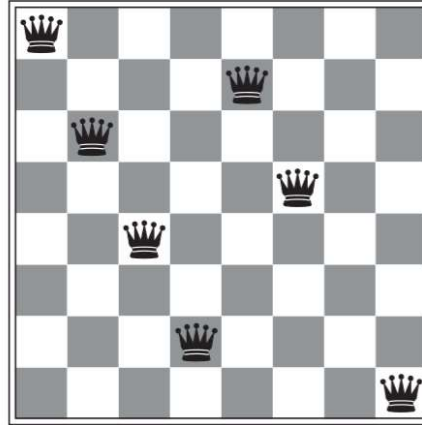
To our knowledge there is no bound on how large a number might be constructed in the process of reaching a given target—for example, the number 620,448,401,733,239,439,360,000 is generated in the expression for 5—so **the state space for this problem is infinite.**

Such state spaces arise frequently in tasks involving the generation of mathematical expressions, circuits, proofs, programs, and other recursively defined objects.

Prof. Dr. Md. Ashikur Rahman Khan

d. 8-queens problem

What is its formulation?



Prof. Dr. Md. Ashikur Rahman Khan

d. 8-queens problem (Cont.)

The first incremental formulation one might try is the following:

States: Any arrangement of 0 to 8 queens on the board is a state.

i. Initial state: No queens on the board.

ii. Actions: Add a queen to any empty square.

iii. Transition model: Returns the board with a queen added to the specified square.

iv. Goal test: 8 queens are on the board such that no queen attacks another.

v. Path costs: 0 (all solutions are equally good) (we are only interested in the solution).

Prof. Dr. Md. Ashikur Rahman Khan

d. 8-queens problem (Contd.)

A better formulation would prohibit placing a queen in any square that is already attacked:

- States: All possible arrangements of n queens ($0 \leq n \leq 8$), one per column in the leftmost n columns, with no queen attacking another.
- Actions: Add a queen to any square in the leftmost empty column such that it is not attacked by any other queen.

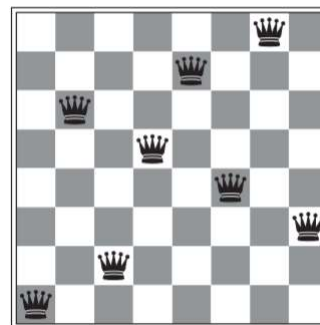
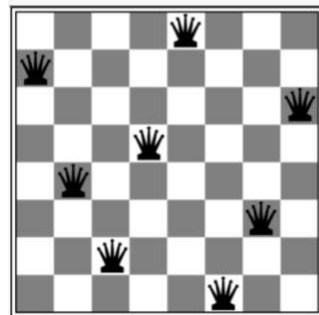
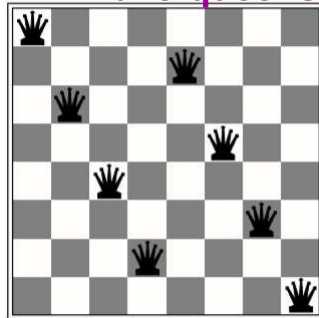
This formulation reduces the 8-queens state space from 1.8×10^{14} to just 2,057, and solutions are easy to find. On the other hand, for 100 queens the reduction is from roughly 10400 states to about 1052 states (Exercise 3.5)—a big improvement, but not enough to make the problem tractable.

Section 4.1 describes the complete-state formulation, and Chapter 6 gives a simple algorithm that solves even the million-queens problem with ease.

d. 8-queens problem (Contd.)

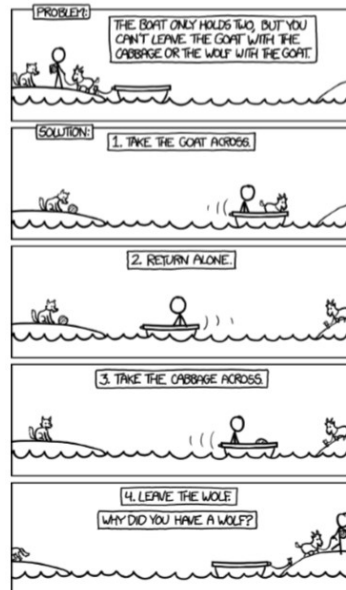


Almost a solution



Prof. Dr. Md. Ashikur Rahman Khan

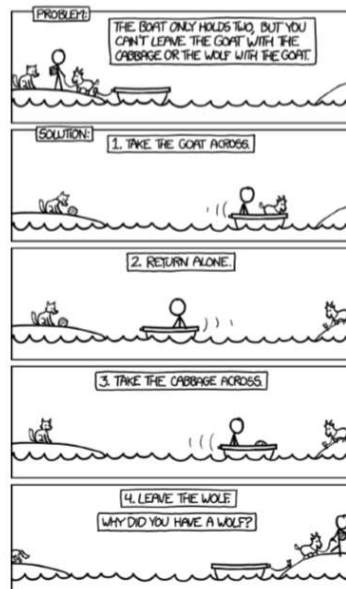
e. Fox, Goat, and Cabbage Fox, Goat, Cabbage



What is its formulation?

Prof. Dr. Md. Ashikur Rahman Khan

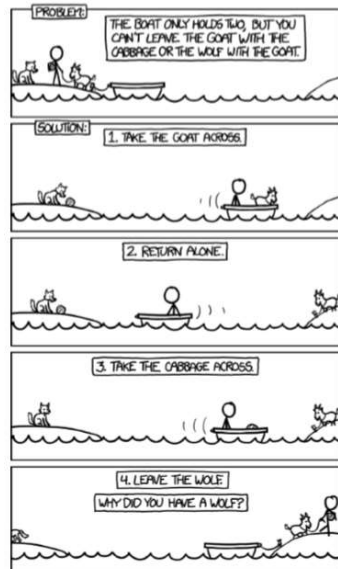
e. Fox, Goat, and Cabbage (Contd.) Fox, Goat, Cabbage



- States
- Actions
- Goal
- Cost

Prof. Dr. Md. Ashikur Rahman Khan

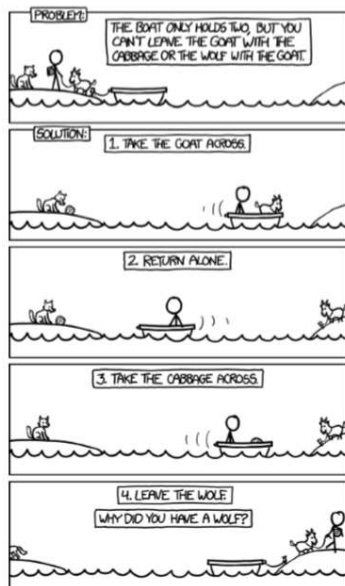
e. Fox, Goat, and Cabbage (Contd.)



- States:
 - Location of fox, goat, cabbage (left bank, boat, right bank) and boat (left, right)
 - Initial state: (left, left, left, left)
- Actions
- Goal
- Cost

Prof. Dr. Md. Ashikur Rahman Khan

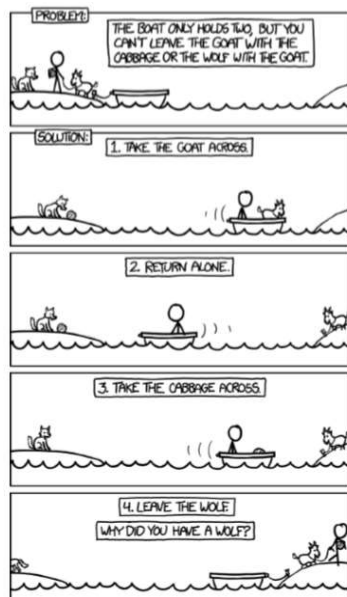
e. Fox, Goat, and Cabbage (Contd.)



- States:
 - Location of fox, goat, cabbage (left bank, boat, right bank) and boat (left, right)
 - Initial state: (left, left, left, left)
- Actions:
 - Boat left, boat right
 - Load fox, unload fox
 - Load goat, unload goat
 - Load cabbage, unload cabbage
 - (never leaving fox with goat, goat with cabbage alone)
- Goal:
- Cost:

Prof. Dr. Md. Ashikur Rahman Khan

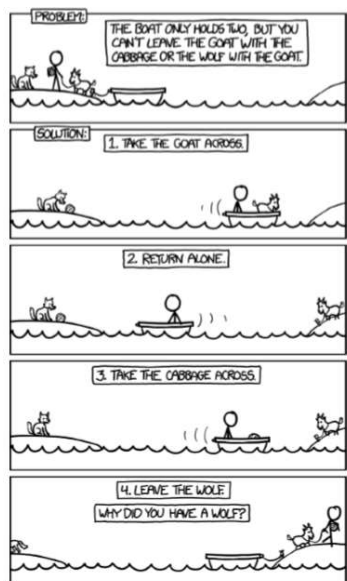
e. Fox, Goat, and Cabbage (Contd.)



- States:
 - Location of fox, goat, cabbage (left bank, boat, right bank) and boat (left, right)
 - Initial state: (left, left, left, left)
- Actions:
 - Boat left, boat right
 - Load fox, unload fox
 - Load goat, unload goat
 - Load cabbage, unload cabbage
 - (never leaving fox with goat, goat with cabbage alone)
- Goal:
 - (right, right, right, right)
- Cost:

Prof. Dr. Md. Ashikur Rahman Khan

e. Fox, Goat, and Cabbage (Contd.)



- States:
 - Location of fox, goat, cabbage (left bank, boat, right bank) and boat (left, right)
 - Initial state: (left, left, left, left)
- Actions:
 - Boat left, boat right
 - Load fox, unload fox
 - Load goat, unload goat
 - Load cabbage, unload cabbage
 - (never leaving fox with goat, goat with cabbage alone)
- Goal:
 - (right, right, right, right)
- Cost:
 - 1 per action

Prof. Dr. Md. Ashikur Rahman Khan

e. Fox, Goat, and Cabbage (Contd.)

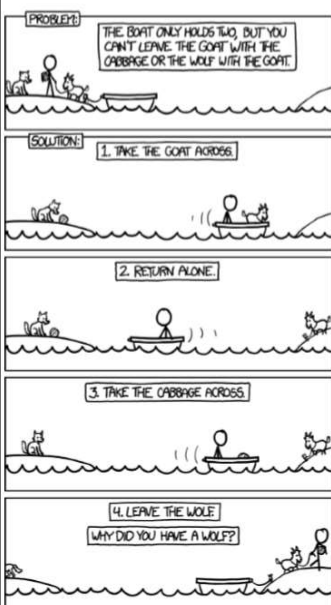


- States:
 - Location: boat, right bank, and boat (left, right)
 - Initial state: (left, left, left, left)
- Actions:
 - Boat left, boat right
 - Load fox, unload fox
 - Load goat, unload goat
 - Load cabbage, unload cabbage
 - (never leaving fox with goat, goat with cabbage alone)
- Goal:
 - (right, right, right, right)
- Cost:
 - 1 per action

What's the size of the state space?

Prof. Dr. Md. Ashikur Rahman Khan

e. Fox, Goat, and Cabbage (Contd.)



- States:
 - Location: boat, right bank, and boat (left, right)
 - Initial state: (left, left, left, left)
- Actions:
 - Boat left, boat right
 - Load fox, unload fox
 - Load goat, unload goat
 - Load cabbage, unload cabbage
 - (never leaving fox with goat, goat with cabbage alone)
- Goal:
 - (right, right, right, right)
- Cost:
 - 1 per action

What's the size of the state space?
At most $3 \times 3 \times 3 \times 2 = 54$ (less when we apply the action constraints)

ii. Real-World Problem

We have already seen how the route-finding problem is defined in terms of specified locations and transitions along links between them.

- **Route-finding algorithms** are used in a variety of applications.
- Some, such as **Web sites and in-car systems** that provide driving directions, are relatively straightforward extensions of the **Romania example**.
- **Others,**
 - **such as routing video streams in computer networks,**
 - **military operations planning, and**
 - **airline travel-planning systems, involve much more complex specifications.**

1. Airline travel-planning systems

Consider the airline travel problems that must be solved by a travel-planning Web site:

States: Each state obviously includes a **location** (e.g., an airport) and the **current time**.

Furthermore, because the **cost** of an action (a flight segment) may depend on previous segments, their fare bases, and their **status as domestic or international**, the state must record extra information about these “historical” aspects.

- **Initial state:** This is specified by the user’s query.
- **Actions:** Take any flight from the current location,
 - in any seat class,
 - leaving after the current time,
 - leaving enough time for within-airport transfer if needed.

1. Airline travel-planning systems (Contd.)

- **Transition model:** The state resulting from taking a flight will have the **flight's destination as the current location** and the **flight's arrival time as the current time**.
- **Goal test:** Are we at the final destination specified by the user?
- **Path cost:** This depends on
 - monetary cost,
 - waiting time,
 - flight time,
 - customs and immigration procedures,
 - seat quality,
 - time of day,
 - type of airplane,
 - frequent-flyer mileage awards,
 - and so on.

Problem-Solving Agent



3. Search

The process of finding a solution is called search.

The process of looking for a sequence of actions that reaches the goal is called search. **A search algorithm takes a problem as input and returns a solution in the form of an action sequence.**

After formulating a goal and a problem to solve, **the agent calls a search** procedure to solve it. It then uses the solution to guide its actions, doing whatever the solution recommends as the next thing to do—typically, the first action of the sequence—and then removing that step from the sequence.

Once a solution is found, the actions it recommends can be carried out. This is called the **execution phase**.

Once the solution has been executed, the agent will formulate a new goal.

Prof. Dr. Md. Ashikur Rahman Khan

Well-defined problems and solutions

The step cost of taking action a in state s to reach state s' is denoted by $c(s, a, s')$. The step costs for Romania are shown in Figure 3.2 as route distances. We assume that step costs are nonnegative.

The preceding elements define a problem and can be gathered into a single data structure that is given as input to a problem-solving algorithm.

A **solution** to a problem is an action sequence that leads from the initial state to a goal state.

Solution quality is measured by the path cost function, and an **optimal solution** has the lowest path cost among all solutions.

Solution - path from initial state to a goal state.

Optimal solution - lowest path cost of all solutions.

Problem-Solving Agents



Problem-solving agent is a one kind of goal-based agent.

We then describe **several general-purpose search algorithms that can be used to solve these problems.**

Such as

- ✓ **Uninformed search algorithms**
- ✓ **Informed search algorithms**

Solving Problems by Searching

In which we see how an agent can find a sequence of actions that achieves its goals when no single action will do.

The simplest agents (as discussed in previous lecture 5) is the reflex agents, which base their actions on a direct mapping from states to actions.

Such agents cannot operate well in environments for which this mapping would be too large to store and would take too long to learn.

On the other hand, Goal-based agents consider future actions and the desirability of their outcomes.

Real World Problem



Airline Travel Problems

Commercial travel advice systems use a problem formulation of this kind, with many additional complications to handle the byzantine fare structures that airlines impose. Any seasoned traveler knows, however, that not all air travel goes according to plan.

A really good system should include contingency plans—such as backup reservations on alternate flights— to the extent that these are justified by the cost and likelihood of failure of the original plan.

Touring problems



Touring problems are closely related to route-finding problems, but with an important difference. Consider, for example, the problem “Visit every city in Figure 3.2 at least once, starting and ending in Bucharest.” As with route finding, the actions correspond to trips between adjacent cities.

The state space, however, is quite different.

Each state must include not just the current location but also the set of cities the agent has visited.

So the initial state would be $In(Bucharest), Visited(\{Bucharest\})$, a typical intermediate state would be $In(Vaslui), Visited(\{Bucharest, Urziceni, Vaslui\})$, and the goal test would check whether the agent is in Bucharest and all 20 cities have been visited.

Prof. Dr. Md. Ashikur Rahman Khan

Traveling salesperson problem



The **traveling salesperson problem** (TSP) is a touring problem in which each city must be visited exactly once. The aim is to find the shortest tour. The problem is known to be NP-hard, but an enormous amount of effort has been expended to improve the capabilities of TSP algorithms. In addition to planning trips for traveling salespersons, these algorithms have been used for tasks such as planning movements of automatic circuit-board drills and of stocking machines on shop floors.

VLSI Layout Problem



A **VLSI layout problem** requires positioning millions of components and connections on a chip to minimize area, minimize circuit delays, minimize stray capacitances, and maximize manufacturing yield. The layout problem comes after the logical design phase and is usually split into two parts: **cell layout and channel routing**. In cell layout, the primitive components of the circuit are grouped into cells, each of which performs some recognized function. Each cell has a fixed footprint (size and shape) and requires a certain number of connections to each of the other cells. The aim is to place the cells on the chip so that they do not overlap and so that there is room for the connecting wires to be placed between the cells. Channel routing finds a specific route for each wire through the gaps between the cells. These search problems are extremely complex, but definitely worth solving. Later in this chapter, we present some algorithms capable of solving them.

Robot navigation

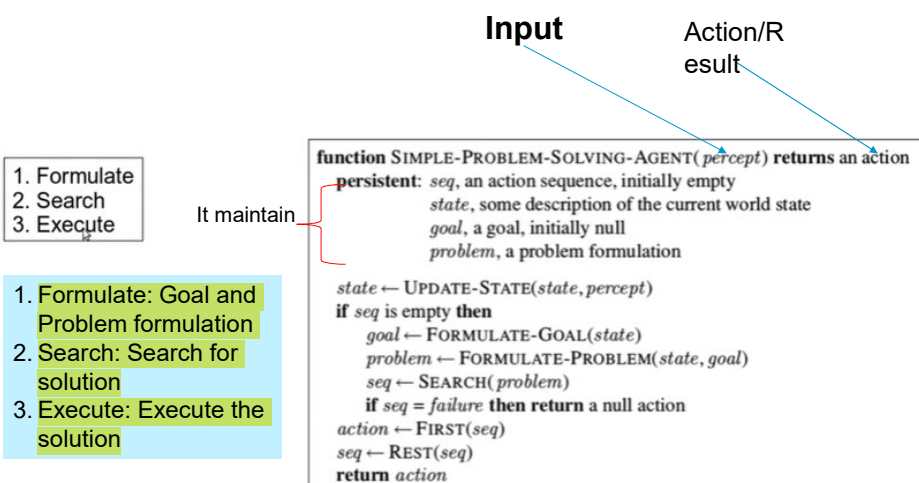


Robot navigation is a generalization of the route-finding problem described earlier. Rather than following a discrete set of routes, **a robot can move in a continuous space with (in principle) an infinite set of possible actions and states**. For a circular robot moving on a flat surface, the space is essentially two-dimensional. When the robot has arms and legs or wheels that must also be controlled, the search space becomes many-dimensional. Advanced techniques are required just to make the search space finite. We examine some of these methods in Chapter 25. In addition to the complexity of the problem, real robots must also deal with errors in their sensor readings and motor controls.



Automatic assembly sequencing of complex objects by a robot was first demonstrated (Michie, 1972). Progress since then has been slow but sure, to the point where the assembly of intricate objects such as electric motors is economically feasible. In assembly problems, the aim is to find an order in which to assemble the parts of some object. If the wrong order is chosen, there will be no way to add some part later in the sequence without undoing some of the work already done. Checking a step in the sequence for feasibility is a difficult geometrical search problem closely related to robot navigation. Thus, the generation of legal actions is the expensive part of assembly sequencing. Any practical algorithm must avoid exploring all but a tiny fraction of the state space. Another important assembly problem is protein design, in which the goal is to find a sequence of amino acids that will fold into a PROTEIN DESIGN three-dimensional protein with the right properties to cure some disease.

Pseudocode of the Problem Solving Agent



Pseudocode of the Problem Solving Agent (Contd.)

1. Formulate
2. Search
3. Execute

Problem formulation is the process of deciding what actions and states to consider, given a goal.

```

function SIMPLE-PROBLEM-SOLVING-AGENT(percept) returns an action


persistent: seq, an action sequence, initially empty  

state, some description of the current world state  

goal, a goal, initially null  

problem, a problem formulation

state ← UPDATE-STATE(state, percept)
if seq is empty then
  goal ← FORMULATE-GOAL(state)
  problem ← FORMULATE-PROBLEM(state, goal)
  seq ← SEARCH(problem)
  if seq = failure then return a null action
action ← FIRST(seq)
seq ← REST(seq)
return action
  
```

Pseudocode of the Problem Solving Agent (Contd.)

1. Formulate
2. Search
3. Execute

Problem formulation is the process of deciding what actions and states to consider, given a goal.

The process of looking for a sequence of actions that reaches the goal is called **search**.

A **search algorithm** takes a problem as input and returns a solution in the form of an action sequence

```

function SIMPLE-PROBLEM-SOLVING-AGENT(percept) returns an action


persistent: seq, an action sequence, initially empty  

state, some description of the current world state  

goal, a goal, initially null  

problem, a problem formulation

state ← UPDATE-STATE(state, percept)
if seq is empty then
  goal ← FORMULATE-GOAL(state)
  problem ← FORMULATE-PROBLEM(state, goal)
  seq ← SEARCH(problem)
  if seq = failure then return a null action
action ← FIRST(seq)
seq ← REST(seq)
return action
  
```

Pseudocode of the Problem Solving Agent (Contd.)

1. Formulate
2. Search
3. Execute

Problem formulation is the process of deciding what actions and states to consider, given a goal.

The process of looking for a sequence of actions that reaches the goal is called **search**.

A **search algorithm** takes a problem as input and returns a solution in the form of an action sequence

```
function SIMPLE-PROBLEM-SOLVING-AGENT(percept) returns an action
  persistent: seq, an action sequence, initially empty
               state, some description of the current world state
               goal, a goal, initially null
               problem, a problem formulation

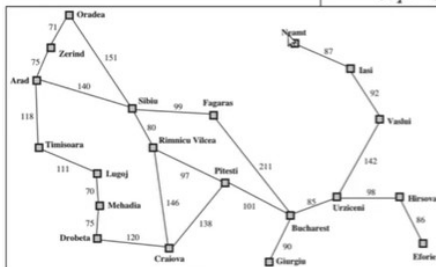
  state ← UPDATE-STATE(state, percept)
  if seq is empty then
    goal ← FORMULATE-GOAL(state)
    problem ← FORMULATE-PROBLEM(state, goal)
    seq ← SEARCH(problem)
    if seq = failure then return a null action
  action ← FIRST(seq)
  seq ← REST(seq)
  return action
```

Once a solution is found, the actions it recommends can be carried out. This is called the **execution phase**.

Description of PSA Pseudocode with an Example

```
function SIMPLE-PROBLEM-SOLVING-AGENT(percept) returns an action
  persistent: seq, an action sequence, initially empty
               state, some description of the current world state
               goal, a goal, initially null
               problem, a problem formulation

  state ← UPDATE-STATE(state, percept)
  if seq is empty then
    goal ← FORMULATE-GOAL(state)
    problem ← FORMULATE-PROBLEM(state, goal)
    seq ← SEARCH(problem)
    if seq = failure then return a null action
  action ← FIRST(seq)
  seq ← REST(seq)
  return action
```



Pseudocode of the Problem Solving Agent (Contd.)

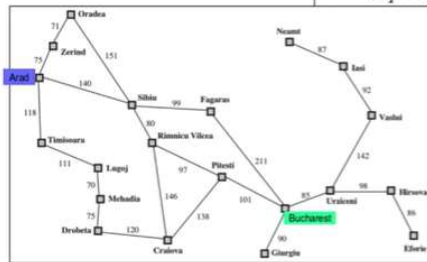
Percept
GPS: Arad
Passenger input: Bucharest

```

function SIMPLE-PROBLEM-SOLVING-AGENT(percept) returns an action
  persistent: seq, an action sequence, initially empty
               state, some description of the current world state
               goal, a goal, initially null
               problem, a problem formulation

  state ← UPDATE-STATE(state, percept)
  if seq is empty then
    goal ← FORMULATE-GOAL(state)
    problem ← FORMULATE-PROBLEM(state, goal)
    seq ← SEARCH(problem)
  failure then return a null action
  RST(seq)
  (seq)
  m

```



Pseudocode of the Problem Solving Agent (Contd.)

Percept
GPS: Arad
Passenger input: Bucharest

```

function SIMPLE-PROBLEM-SOLVING-AGENT(percept) returns an action
  persistent: seq, an action sequence, initially empty
               state, some description of the current world state
               goal, a goal, initially null
               problem, a problem formulation

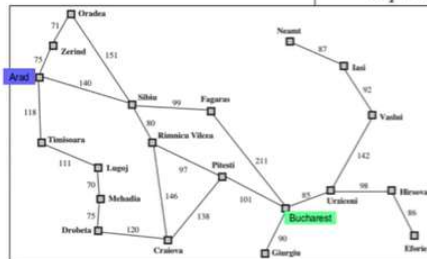
  state ← UPDATE-STATE(state, percept)
  if seq is empty then
    goal ← FORMULATE-GOAL(state)
    problem ← FORMULATE-PROBLEM(state, goal)
    seq ← SEARCH(problem)

```

```

  failure then return a null action
  RST(seq)
  (seq)
  m

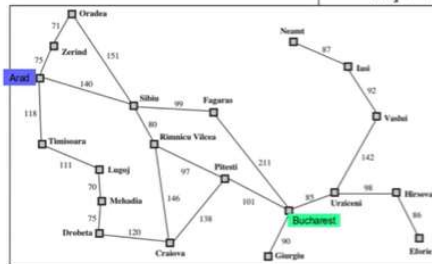
```



Pseudocode of the Problem Solving Agent (Contd.)

Percept
GPS: Arad
Passenger input: Bucharest

Goal: Bucharest



function SIMPLE-PROBLEM-SOLVING-AGENT(*percept*) **returns** an action
persistent: *seq*, an action sequence, initially empty
state, some description of the current world state
goal, a goal, initially null
problem, a problem formulation

state ← UPDATE-STATE(*state*, *percept*)

if *seq* is empty **then**

goal ← FORMULATE-GOAL(*state*)

problem ← FORMULATE-PROBLEM(*state*, *goal*)

seq ← SEARCH(*problem*)

failure then return a null action

RST(*seq*)

(*seq*)

m

Pseudocode of the Problem Solving Agent (Contd.)

Percept
GPS: Arad
Passenger input: Bucharest

Goal: Bucharest

Initial State: In(Arad)



function SIMPLE-PROBLEM-SOLVING-AGENT(*percept*) **returns** an action
persistent: *seq*, an action sequence, initially empty
state, some description of the current world state
goal, a goal, initially null
problem, a problem formulation

state ← UPDATE-STATE(*state*, *percept*)

if *seq* is empty **then**

goal ← FORMULATE-GOAL(*state*)

problem ← FORMULATE-PROBLEM(*state*, *goal*)

seq ← SEARCH(*problem*)

failure then return a null action

RST(*seq*)

(*seq*)

m

Pseudocode of the Problem Solving Agent (Contd.)

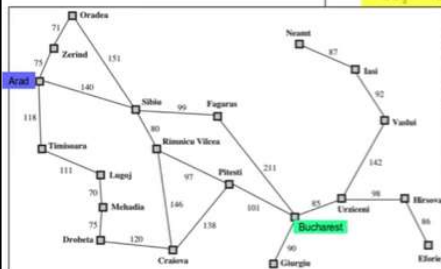
Percept

GPS: Arad

Passenger input: Bucharest

Goal: Bucharest

Initial State: In(Arad)



function SIMPLE-PROBLEM-SOLVING-AGENT(*percept*) **returns** an action

persistent: *seq*, an action sequence, initially empty

state, some description of the current world state

goal, a goal, initially null

problem, a problem formulation

state $\leftarrow$ UPDATE-STATE(*state*, *percept*)

if *seq* is empty **then**

goal $\leftarrow$ FORMULATE-GOAL(*state*)

problem $\leftarrow$ FORMULATE-PROBLEM(*state*, *goal*)

seq $\leftarrow$ SEARCH(*problem*)

failure then return a null action

REST(*seq*)

(*seq*)

n

seq = [Go(Sibiu), Go(Fagaras), Go(Bucharest)]

Pseudocode of the Problem Solving Agent (Contd.)

Percept

GPS: Arad

Passenger input: Bucharest

Goal: Bucharest

Initial State: In(Arad)

function SIMPLE-PROBLEM-SOLVING-AGENT(*percept*) **returns** an action

persistent: *seq*, an action sequence, initially empty

state, some description of the current world state

goal, a goal, initially null

problem, a problem formulation

state $\leftarrow$ UPDATE-STATE(*state*, *percept*)

if *seq* is empty **then**

goal $\leftarrow$ FORMULATE-GOAL(*state*)

problem $\leftarrow$ FORMULATE-PROBLEM(*state*, *goal*)

seq $\leftarrow$ SEARCH(*problem*)

if *seq* = failure **then return** a null action

action $\leftarrow$ FIRST(*seq*)

seq $\leftarrow$ REST(*seq*)

return *action*

seq = [Go(Sibiu), Go(Fagaras), Go(Bucharest)]

Action 1: Go(Sibiu)

Pseudocode of the Problem Solving Agent (Contd.)

Percept
GPS: Arad
Passenger input: Bucharest

Goal: Bucharest

Initial State: In(Arad)

```
function SIMPLE-PROBLEM-SOLVING-AGENT(percept) returns an action
  persistent: seq, an action sequence, initially empty
               state, some description of the current world state
               goal, a goal, initially null
               problem, a problem formulation

  state ← UPDATE-STATE(state, percept)
  if seq is empty then
    goal ← FORMULATE-GOAL(state)
    problem ← FORMULATE-PROBLEM(state, goal)
    seq ← SEARCH(problem)
    if seq = failure then return a null action
  action ← FIRST(seq)
  seq ← REST(seq)
  return action
```

seq = [Go(Sibiu), Go(Fagaras), Go(Bucharest)]

Action 1: Go(Sibiu)

seq = [Go(Fagaras), Go(Bucharest)]

Pseudocode of the Problem Solving Agent (Contd.)

Percept
GPS: Arad
Passenger input: Bucharest

Goal: Bucharest

Initial State: In(Arad)

```
function SIMPLE-PROBLEM-SOLVING-AGENT(percept) returns an action
  persistent: seq, an action sequence, initially empty
               state, some description of the current world state
               goal, a goal, initially null
               problem, a problem formulation

  state ← UPDATE-STATE(state, percept)
  if seq is empty then
    goal ← FORMULATE-GOAL(state)
    problem ← FORMULATE-PROBLEM(state, goal)
    seq ← SEARCH(problem)
    if seq = failure then return a null action
  action ← FIRST(seq)
  seq ← REST(seq)
  return action
```

seq = [Go(Sibiu), Go(Fagaras), Go(Bucharest)]

Action 1: Go(Sibiu)

seq = [Go(Fagaras), Go(Bucharest)]

Action 2: Go(Fagaras)

Now, since the *seq* is not empty avoid the if condition and go to action directly

Pseudocode of the Problem Solving Agent (Contd.)

Percept
GPS: Arad
Passenger input: Bucharest

Goal: Bucharest

Initial State: In(Arad)

```
function SIMPLE-PROBLEM-SOLVING-AGENT(percept) returns an action
  persistent: seq, an action sequence, initially empty
               state, some description of the current world state
               goal, a goal, initially null
               problem, a problem formulation

  state ← UPDATE-STATE(state, percept)
  if seq is empty then
    goal ← FORMULATE-GOAL(state)
    problem ← FORMULATE-PROBLEM(state, goal)
    seq ← SEARCH(problem)
    if seq = failure then return a null action
  action ← FIRST(seq)
  seq ← REST(seq)
  return action
```

seq = [Go(Sibiu), Go(Fagaras), Go(Bucharest)]

Action 1: Go(Sibiu)

seq = [Go(Fagaras), Go(Bucharest)]

Action 2: Go(Fagaras)

seq = [Go(Bucharest)]

Pseudocode of the Problem Solving Agent (Contd.)

Percept
GPS: Arad
Passenger input: Bucharest

Goal: Bucharest

Initial State: In(Arad)

```
function SIMPLE-PROBLEM-SOLVING-AGENT(percept) returns an action
  persistent: seq, an action sequence, initially empty
               state, some description of the current world state
               goal, a goal, initially null
               problem, a problem formulation

  state ← UPDATE-STATE(state, percept)
  if seq is empty then
    goal ← FORMULATE-GOAL(state)
    problem ← FORMULATE-PROBLEM(state, goal)
    seq ← SEARCH(problem)
    if seq = failure then return a null action
  action ← FIRST(seq)
  seq ← REST(seq)
  return action
```

seq = [Go(Sibiu), Go(Fagaras), Go(Bucharest)]

Action 1: Go(Sibiu)

seq = [Go(Fagaras), Go(Bucharest)]

Action 2: Go(Fagaras)

seq = [Go(Bucharest)]

Action 3: Go(Bucharest)

Pseudocode of the Problem Solving Agent (Contd.)

Percept
GPS: Arad
Passenger input: Bucharest

Goal: Bucharest

Initial State: In(Arad)

```
function SIMPLE-PROBLEM-SOLVING-AGENT(percept) returns an action
  persistent: seq, an action sequence, initially empty
               state, some description of the current world state
               goal, a goal, initially null
               problem, a problem formulation

  state ← UPDATE-STATE(state, percept)
  if seq is empty then
    goal ← FORMULATE-GOAL(state)
    problem ← FORMULATE-PROBLEM(state, goal)
    seq ← SEARCH(problem)
    if seq = failure then return a null action
  action ← FIRST(seq)
  seq ← REST(seq)
  return action
```

seq = [Go(Sibiu), Go(Fagaras), Go(Bucharest)]

Action 1: Go(Sibiu)

seq = [Go(Fagaras), Go(Bucharest)]

Action 2: Go(Fagaras)

seq = [Go(Bucharest)]

Action 3: Go(Bucharest)

seq = []

Pseudocode of the Problem Solving Agent (Contd.)

```
function SIMPLE-PROBLEM-SOLVING-AGENT(percept) returns an action
  persistent: seq, an action sequence, initially empty
               state, some description of the current world state
               goal, a goal, initially null
               problem, a problem formulation

  state ← UPDATE-STATE(state, percept)
  if seq is empty then
    goal ← FORMULATE-GOAL(state)
    problem ← FORMULATE-PROBLEM(state, goal)
    seq ← SEARCH(problem)
    if seq = failure then return a null action
  action ← FIRST(seq)
  seq ← REST(seq)
  return action
```

This part is not entered during the execute stage. During the execute stage, even the new percepts are ignored.

